



Informe de resultados de pruebas

Aprendiz – Juan Pablo Mejia Vargas CC 1012359571

Servicio Nacional de Aprendizaje SENA

Centro de Formación en Diseño, Confección y Moda

Tecnología en Análisis y Desarrollo de Software – ADSO

Ficha: 2977373



1. RESUMEN EJECUTIVO

Indicador	Valor
Total de casos de prueba definidos	56
Casos ejecutados	56
Casos exitosos	54
Casos omitidos (SMTP no configurado)	2
Cobertura de módulos	100% (Auth, Usuarios, Citas)
Herramienta de automatización	Cypress 15.x
Ambiente	Producción - HTTPS

Un software de prueba automatizado está controlando Chrome

segura-mente-app master

Specs

Search specs 11 matches + New spec

L2E specs Component specs

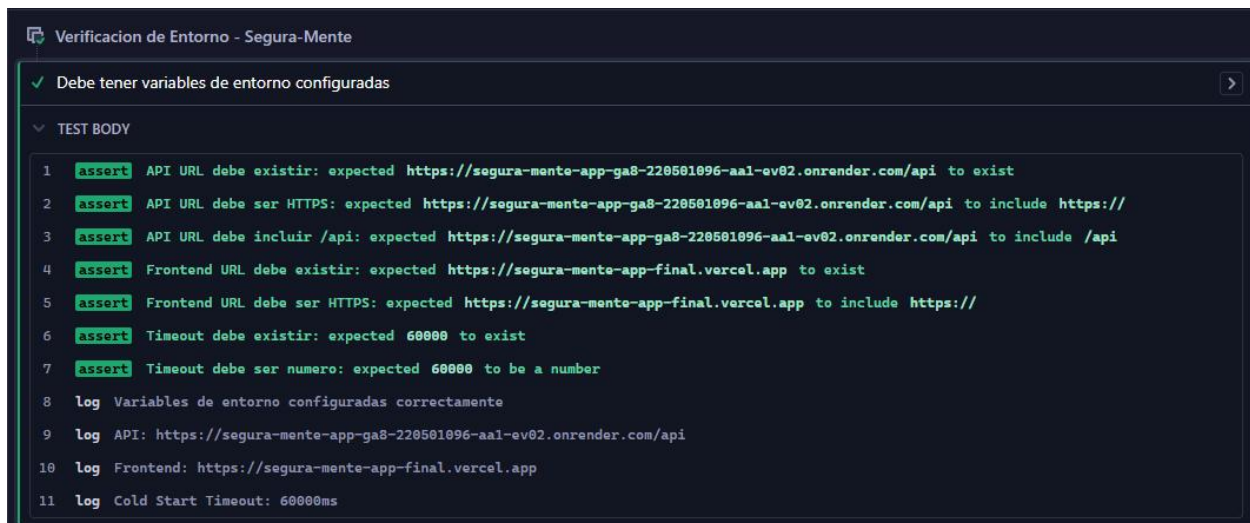
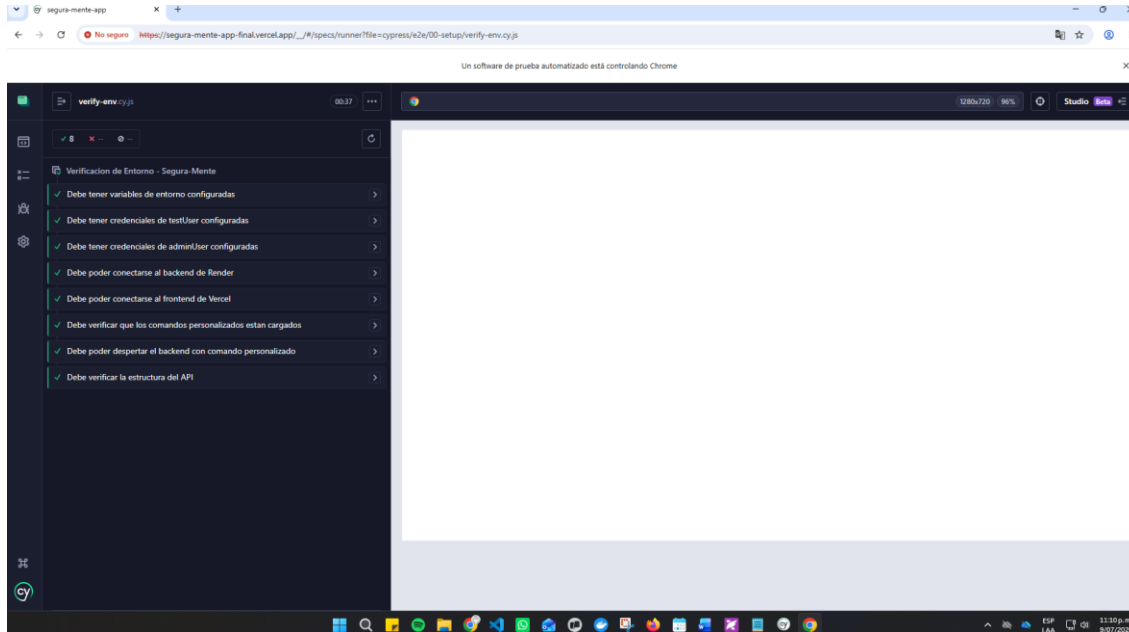
Last updated Latest runs Average duration

- 00-setup
 - verify-env cypress 9 days ago
- 01-auth
 - forgot-password cypress 31 minutes ago
 - login cypress 9 days ago
 - register cypress 31 mins ago
 - resend-verification cypress 31 minutes ago
- 02-users
 - create-user cypress 9 days ago
 - delete-user cypress 9 days ago
 - get-user cypress 9 days ago
 - list-users cypress 9 days ago
 - update-user cypress 9 days ago
- 03-appointments
 - appointments cypress 31 minutes ago



2. EJECUCIÓN DE PRUEBAS EN CYPRESS

Verificación inicial de entorno:





✓ Debe tener credenciales de testUser configuradas

TEST BODY

```
1  assert testUser debe existir: expected { Object (email, password, ...) } to exist
2  assert testUser.email debe existir: expected cliente.prueba@correo.com to exist
3  assert testUser.password debe existir: expected Admin1234* to exist
4  assert testUser.nombreUsuario debe existir: expected ClientePrueba to exist
5  assert testUser.tipoIdentificacion debe existir: expected CC to exist
6  assert testUser.identificacion debe existir: expected '1000000004' to exist
7  log Credenciales de testUser configuradas
8  log Email: cliente.prueba@correo.com
```

✓ Debe tener credenciales de adminUser configuradas

TEST BODY

```
1  assert adminUser debe existir: expected { Object (email, password, ...) } to exist
2  assert adminUser.email debe existir: expected administrador@seguramente.com to exist
3  assert adminUser.password debe existir: expected Admin1234* to exist
4  log Credenciales de adminUser configuradas
5  log Email: administrador@seguramente.com
```

✓ Debe poder conectarse al backend de Render

TEST BODY

```
1  log Intentando conectar al backend...
2  log NOTA: Primera conexion puede tardar hasta 60 segundos (cold start)
3  request  GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/
4  -assert Backend debe responder: expected [ 200, 404 ] to be an array
5  -assert Backend debe responder: expected 200 to be one of [ 200, 404 ]
6  log Backend respondio con status: 200
7  log Tiempo de respuesta: 175ms
```

✓ Debe poder conectarse al frontend de Vercel

TEST BODY

```
1  log Intentando conectar al frontend...
2  request  GET 200 /
3  -assert Frontend debe responder: expected 200 to equal 200
4  log Frontend respondio con status: 200
5  log Tiempo de respuesta: 648ms
```

✓ Debe verificar que los comandos personalizados estan cargados

TEST BODY

```
1  assert Comando wakeUpBackend debe existir: expected [Function] to exist
2  assert Comando login debe existir: expected [Function] to exist
3  assert Comando apiRequest debe existir: expected [Function] to exist
4  assert Comando registerUser debe existir: expected [Function] to exist
5  assert Comando generateTestUser debe existir: expected [Function] to exist
6  log Todos los comandos personalizados estan cargados
```



```
✓ Debe poder despertar el backend con comando personalizado

TEST BODY

1 log Despertando backend de Render...
2 log Esto puede tardar hasta 90 segundos en cold start
3 request GET 200 https://segura-mente-app-ga8-220501096-aa1-ev02.onrender.com/
4 log Intento 1: Status 200
5 wait 10000
6 request GET 200 https://segura-mente-app-ga8-220501096-aa1-ev02.onrender.com/
7 log Intento 2: Status 200
8 wait 10000
9 request GET 200 https://segura-mente-app-ga8-220501096-aa1-ev02.onrender.com/
10 log Intento 3: Status 200
11 wait 10000
12 request GET 200 https://segura-mente-app-ga8-220501096-aa1-ev02.onrender.com/
13 log Intento 4: Status 200
14 log ✓ Backend activo y respondiendo
15 wait 5000
16 log Backend despertado exitosamente
```

```
✓ Debe verificar la estructura del API

TEST BODY

1 log Verificando endpoints del API...
2 request GET 401 https://segura-mente-app-ga8-220501096-aa1-ev02.onrender.com/api/users
3 -assert Debe requerir autenticacion: expected 401 to equal 401
4 log API requiere autenticacion correctamente
Clear page
```

2.1 Modulo de Autenticación

Registro

ID	Caso en Cypress	Descripción	Tipo	Prioridad	Resultado
CP-01	Caso 1.1	Registro de usuario con datos validos	Funcional / Automatizada	Alta	EXITOSO
CP-02	Caso 1.2	Registro con email duplicado	Funcional / Automatizada	Alta	EXITOSO
CP-04	Caso 1.2b	Registro con identificación duplicada	Funcional / Automatizada	Media	EXITOSO
CP-03	Caso 1.3	Registro con contraseña débil	Funcional / Automatizada	Alta	EXITOSO
CP-04b	Caso 1.4	Registro con campos obligatorios faltantes	Funcional / Automatizada	Alta	EXITOSO
CP-05	Caso 1.5	Registro con edad menor de 18 anos	Funcional / Automatizada	Media	EXITOSO



CP-06	Caso 1.6	Registro con teléfono de formato incorrecto	Funcional / Automatizada	Media	EXITOSO
CP-06b	Caso 1.7	Registro con formato de email invalido	Funcional / Automatizada	Media	EXITOSO
CP-06c	Caso 1.8	Registro con contraseñas que no coinciden	Funcional / Automatizada	Media	EXITOSO
CP-06d	Caso 1.9	Registro de usuario tipo Psicólogo con campos adicionales	Funcional / Automatizada	Alta	EXITOSO
CP-06e	Caso 1.10	Tiempo de respuesta del registro dentro del limite	Rendimiento / Automatizada	Baja	EXITOSO

Un software de prueba automatizado está controlando Chrome

register.cy.js00:41

✓ 11 ✗ 0

Autenticacion - Registro de Usuario

✓ Caso 1.1: Debe registrar un nuevo usuario con datos validos

✓ Caso 1.2: Debe fallar con email duplicado

✓ Caso 1.2b - CP-04: Debe fallar con identificacion duplicada

✓ Caso 1.3: Debe fallar con contrasena debil

✓ Caso 1.4: Debe fallar con campos faltantes

✓ Caso 1.5: Debe fallar con usuario menor de edad

✓ Caso 1.6: Debe fallar con telefono invalido

✓ Caso 1.7: Debe fallar con formato de email invalido

✓ Caso 1.8: Debe validar que las contrasenas coincidan

✓ Caso 1.9: Debe registrar usuario tipo Psicologo con campos adicionales

✓ Caso 1.10: Debe validar tiempo de respuesta aceptable



✓ Caso 1.1: Debe registrar un nuevo usuario con datos válidos

▼ BEFORE ALL

```
1 log Despertando backend de Render...
2 log Despertando backend de Render...
3 log Esto puede tardar hasta 90 segundos en cold start
4 request ○GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/
5 log Intento 1: Status 200
6 wait 10000
7 request ○GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/
8 log Intento 2: Status 200
9 wait 10000
10 request ○GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/
11 log Intento 3: Status 200
12 wait 10000
13 request ○GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/
14 log Intento 4: Status 200
15 log ✓ Backend activo y respondiendo
16 wait 5000
```

▼ TEST BODY

```
1 log Registrando usuario: test1783657961916@example.com
2 log Registrando usuario: test1783657961916@example.com
3 request ○POST 201 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/register
4 -assert Status debe ser 201: expected 201 to equal 201
5 -assert Success debe ser true: expected true to be true
6 -assert Debe tener data: expected { Object (email, nombreUsuario) } to exist
7 -assert Email debe coincidir: expected test1783657961916@example.com to equal test1783657961916@example.com
8 -assert Debe tener mensaje de éxito: expected Usuario registrado exitosamente. Ya puedes iniciar sesión. to include exitosamente
9 -assert Password no debe estar en respuesta: expected undefined to not exist
10 -assert Debe responder en menos de 30 segundos: expected 1023 to be below 30000
11 log Usuario registrado exitosamente en 1023ms
```

✓ Caso 1.2: Debe fallar con email duplicado

▼ TEST BODY

```
1 log Intentando registrar email duplicado: cliente.prueba@correo.com
2 log Registrando usuario: cliente.prueba@correo.com
3 request ○POST 400 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/register
4 -assert Status debe ser 400 o 409: expected [ 400, 409 ] to be an array
5 -assert Status debe ser 400 o 409: expected 400 to be one of [ 400, 409 ]
6 -assert Success debe ser false: expected false to be false
7 -assert Debe mencionar el conflicto de datos: expected [ Array(1) ] to exist
8 log Respuesta del servidor: El número de identificación ya está registrado
9 log Error de email duplicado detectado correctamente
```



✓ Caso 1.2b - CP-04: Debe fallar con identificacion duplicada

✓ TEST BODY

```
1 log Intentando registrar identificacion duplicada: 1000000004
2 log Registrando usuario: test1783657963367@example.com
3 request ○ POST 400 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/register
4 -assert Status debe ser 400 o 409: expected [ 400, 409 ] to be an array
5 -assert Status debe ser 400 o 409: expected 400 to be one of [ 400, 409 ]
6 -assert Success debe ser false: expected false to be false
7 log Error de identificacion duplicada detectado correctamente
```

✓ Caso 1.3: Debe fallar con contraseña debil

✓ TEST BODY

```
1 log Intentando registrar con contraseña debil: 123
2 log Registrando usuario: test1783657963757@example.com
3 request ○ POST 400 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/register
4 -assert Status debe ser 400: expected 400 to equal 400
5 -assert Success debe ser false: expected false to be false
6 -assert Debe tener mensaje de error: expected Errores de validación to exist
7 log Validacion de contraseña debil funciona correctamente
```

✓ Caso 1.4: Debe fallar con campos faltantes

✓ TEST BODY

```
1 log Intentando registrar con datos incompletos
2 log Registrando usuario: incompleto1783657963951@example.com
3 request ○ POST 400 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/register
4 -assert Status debe ser 400: expected 400 to equal 400
5 -assert Success debe ser false: expected false to be false
6 -assert Debe tener errores de validacion: expected [ Array(16) ] to exist
7 log Validacion de campos obligatorios funciona correctamente
```




✓ Caso 1.5: Debe fallar con usuario menor de edad

✓ TEST BODY

```
1 log Intentando registrar menor de edad: 2016-01-01
2 log Registrando usuario: test1783657964144@example.com
3 request ○POST 400 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/register
4 -assert Status debe ser 400: expected 400 to equal 400
5 -assert Success debe ser false: expected false to be false
6 -assert Debe tener mensaje de error: expected Errores de validación to exist
7 log Validacion de edad minima funciona correctamente
```

✓ Caso 1.6: Debe fallar con telefono invalido

✓ TEST BODY

```
1 log Intentando registrar con telefono invalido: 123
2 log Registrando usuario: test1783657964330@example.com
3 request ○POST 400 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/register
4 -assert Status debe ser 400: expected 400 to equal 400
5 -assert Success debe ser false: expected false to be false
6 -assert Debe tener mensaje de error: expected Errores de validación to exist
7 log Validacion de telefono funciona correctamente
```

✓ Caso 1.7: Debe fallar con formato de email invalido

✓ TEST BODY

```
1 log Intentando registrar con email invalido: email-invalido
2 log Registrando usuario: email-invalido
3 request ○POST 400 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/register
4 -assert Status debe ser 400: expected 400 to equal 400
5 -assert Success debe ser false: expected false to be false
6 -assert Debe mencionar formato de email: expected Errores de validación to exist
7 log Validacion de formato de email funciona correctamente
```

✓ Caso 1.8: Debe validar que las contraseñas coincidan

✓ TEST BODY

```
1 log Intentando registrar con contraseñas diferentes
2 log Registrando usuario: test1783657964707@example.com
3 request ○POST 400 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/register
4 -assert Status debe ser 400: expected 400 to equal 400
5 -assert Success debe ser false: expected false to be false
6 -assert Debe tener mensaje de error: expected Errores de validación to exist
7 log Validacion de coincidencia de contraseñas funciona correctamente
```



```
✓ Caso 1.9: Debe registrar usuario tipo Psicologo con campos adicionales

TEST BODY

1 log Registrando psicologo: test1783657964892@example.com
2 log Registrando usuario: test1783657964892@example.com
3 request POST 201 https://segura-mente-app-ga8-228501096-aal-ev02.onrender.com/api/auth/register
4 -assert Status debe ser 201: expected 201 to equal 201
5 -assert Success debe ser true: expected true to be true
6 -assert Email debe coincidir: expected test1783657964892@example.com to equal test1783657964892@example.com
7 log Psicologo registrado exitosamente
```

```
✓ Caso 1.10: Debe validar tiempo de respuesta aceptable

TEST BODY

1 log Registrando usuario: test1783657965850@example.com
2 request POST 201 https://segura-mente-app-ga8-228501096-aal-ev02.onrender.com/api/auth/register
3 -assert Debe responder en menos de 30 segundos: expected 1002 to be below 30000
4 log Tiempo de respuesta: 1002ms

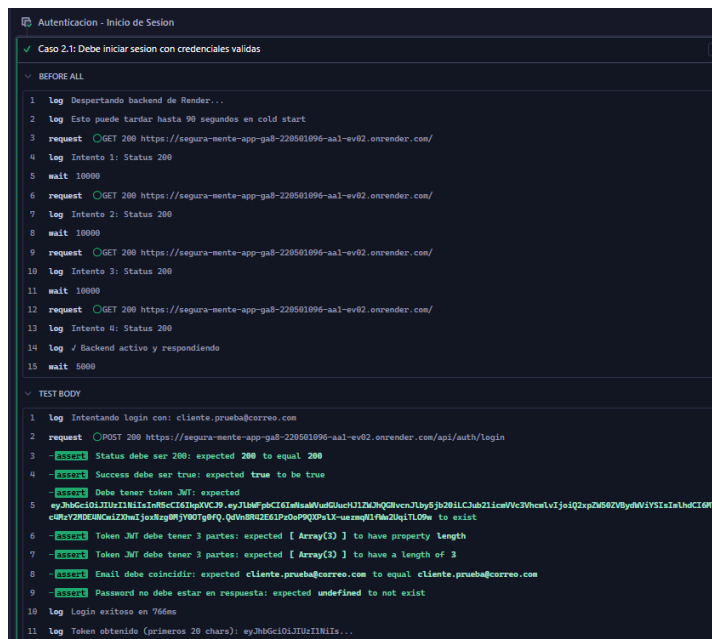
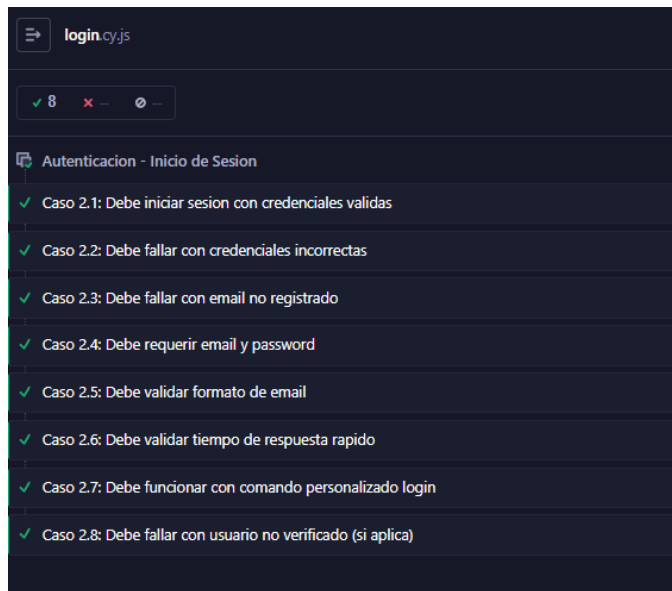
Clear page
```

Login

ID	Caso en Cypress	Descripción	Tipo	Prioridad	Resultado
CP-07	Caso 2.1	Inicio de sesión con credenciales validas y token JWT	Funcional / Automatizada	Alta	EXITOSO
CP-08	Caso 2.2	Inicio de sesión con contraseña incorrecta	Funcional / Automatizada	Alta	EXITOSO
CP-09	Caso 2.3	Inicio de sesión con email no registrado	Funcional / Automatizada	Alta	EXITOSO
CP-10	Caso 2.4	Inicio de sesión con campos vacíos	Funcional / Automatizada	Media	EXITOSO
CP-10b	Caso 2.5	Inicio de sesión con formato de email invalido	Funcional / Automatizada	Media	EXITOSO
CP-10c	Caso 2.6	Tiempo de respuesta del login dentro del limite	Rendimiento / Automatizada	Baja	EXITOSO



CP-10d	Caso 2.7	Comando personalizado de login retorna token valido	Integración / Automatizada	Media	EXITOSO
CP-10e	Caso 2.8	Comportamiento con usuario no verificado	Funcional / Automatizada	Media	EXITOSO





✓ Caso 2.2: Debe fallar con credenciales incorrectas

TEST BODY

```
1 log Intentando login con password incorrecta
2 request ○ POST 404 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/login
3 -assert Status debe ser 401 o 404: expected [ 401, 404 ] to be an array
4 -assert Status debe ser 401 o 404: expected 404 to be one of [ 401, 404 ]
5 -assert Success debe ser false: expected false to be false
6 -assert No debe devolver token: expected undefined to not exist
7 log Error de credenciales incorrectas detectado correctamente
```

✓ Caso 2.3: Debe fallar con email no registrado

TEST BODY

```
1 log Intentando login con email no registrado: noexiste1783660184287@example.com
2 request ○ POST 404 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/login
3 -assert Status debe ser 401 o 404: expected [ 401, 404 ] to be an array
4 -assert Status debe ser 401 o 404: expected 404 to be one of [ 401, 404 ]
5 -assert Success debe ser false: expected false to be false
6 log Error de email no registrado detectado correctamente
```

✓ Caso 2.4: Debe requerir email y password

TEST BODY

```
1 log Intentando login sin credenciales
2 request ○ POST 400 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/login
3 -assert Status debe ser 400: expected 400 to equal 400
4 -assert Success debe ser false: expected false to be false
5 log Validacion de campos requeridos funciona correctamente
```

✓ Caso 2.5: Debe validar formato de email

TEST BODY

```
1 log Intentando login con formato de email invalido
2 request ○ POST 400 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/login
3 -assert Status debe ser 400: expected 400 to equal 400
4 -assert Success debe ser false: expected false to be false
5 log Validacion de formato de email funciona correctamente
```

✓ Caso 2.6: Debe validar tiempo de respuesta rapido

TEST BODY

```
1 request ○ POST 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/login
2 -assert Login debe responder en menos de 5 segundos: expected 777 to be below 5000
3 log Tiempo de respuesta: 777ms
```

✓ Caso 2.7: Debe funcionar con comando personalizado login

TEST BODY

```
1 log Probando comando personalizado cy.login()
2 request ○ POST 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/login
3 -assert Token debe tener formato JWT: expected [ Array(3) ] to have property length
4 -assert Token debe tener formato JWT: expected [ Array(3) ] to have a length of 3
5 log Comando personalizado cy.login() funciona correctamente
```

✓ Caso 2.8: Debe fallar con usuario no verificado (si aplica)

TEST BODY

```
1 log Registrando usuario: test1783660186507@example.com
2 request ○ POST 201 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/register
3 log Usuario registrado: test1783660186507@example.com
4 request ○ POST 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/login
5 log Sistema permite login sin verificacion - Revisar politicas
```

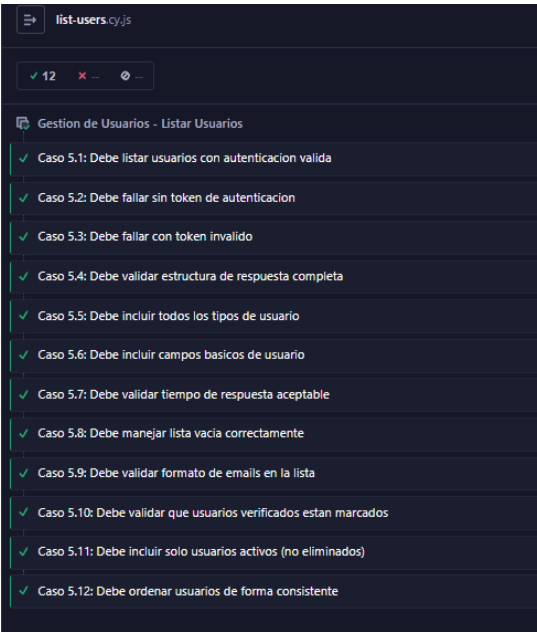
Clear page



2.3 Módulo de gestión de usuarios

Listar

ID	Caso Cypress	Descripción	Tipo	Resultado
CP-11	Caso 5.1	Listar usuarios con autenticación valida	Integración / Auto	EXITOSO
CP-12	Caso 5.2	Fallar sin token de autenticación	Seguridad / Auto	EXITOSO
CP-12b	Caso 5.3	Fallar con token invalido	Seguridad / Auto	EXITOSO
CP-12c	Caso 5.4	Validar estructura completa de respuesta	Integración / Auto	EXITOSO
CP-12d	Caso 5.5	Incluir todos los tipos de usuario en la lista	Funcional / Auto	EXITOSO
CP-12e	Caso 5.6	Incluir campos básicos del usuario	Funcional / Auto	EXITOSO
CP-12f	Caso 5.7	Tiempo de respuesta aceptable	Rendimiento / Auto	EXITOSO
CP-12g	Caso 5.8	Manejar lista vacía correctamente	Funcional / Auto	EXITOSO
CP-12h	Caso 5.9	Validar formato de emails en la lista	Funcional / Auto	EXITOSO
CP-12i	Caso 5.10	Validar que usuarios verificados están marcados	Funcional / Auto	EXITOSO
CP-12j	Caso 5.11	Incluir solo usuarios activos no eliminados	Funcional / Auto	EXITOSO
CP-12k	Caso 5.12	Ordenar usuarios de forma consistente	Funcional / Auto	EXITOSO



```

Gestion de Usuarios - Listar Usuarios

✓ Caso 5.1: Debe listar usuarios con autentificacion valida

▼ BEFORE ALL

1 log Despertando backend de Render...
2 log Esto puede tardar hasta 90 segundos en cold start
3 request GET 200 https://segura-mente-app-ga8-228501096-aal-ev02.onrender.com/
4 log Intento 1: Status 200
5 wait 10000
6 request GET 200 https://segura-mente-app-ga8-228501096-aal-ev02.onrender.com/
7 log Intento 2: Status 200
8 wait 10000
9 request GET 200 https://segura-mente-app-ga8-228501096-aal-ev02.onrender.com/
10 log Intento 3: Status 200
11 wait 10000
12 request GET 200 https://segura-mente-app-ga8-228501096-aal-ev02.onrender.com/
13 log Intento 4: Status 200
14 log ✓ Backend activo y respondiendo
15 wait 5000

▼ TEST BODY

1 log Iniciando sesion como: testUser
2 request POST 200 https://segura-mente-app-ga8-228501096-aal-ev02.onrender.com/api/auth/Login
3 log Login exitoso. Token obtenido.
4 log Consultando lista de usuarios...
5 log GET /users
6 request GET 200 https://segura-mente-app-ga8-228501096-aal-ev02.onrender.com/api/users
7 -assert Status debe ser 200: expected 200 to equal 200
8 -assert Success debe ser true: expected true to be true
9 -assert Debe tener lista de usuarios: expected [ Array(30) ] to exist
10 -assert Lista debe ser un array: expected true to be true
11 -assert Usuario debe tener email: expected { Object (email, nombre_usuario, ...) } to have property email
12 -assert Usuario debe tener nombre: expected { Object (email, nombre_usuario, ...) } to have property nombre_usuario
13 -assert Usuario debe tener tipo: expected { Object (email, nombre_usuario, ...) } to have property tipo_usuario
14 -assert Password no debe estar expuesta: expected undefined to not exist
15 -assert Password hash no debe estar expuesto: expected undefined to not exist
16 log Total de usuarios encontrados: 30
17 log Estructura de datos validada correctamente

```



✓ Caso 5.2: Debe fallar sin token de autenticacion

✓ TEST BODY

```
1 log Intentando listar usuarios sin autenticacion
2 request ○GET 401 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/users
3 -assert Status debe ser 401 o 403: expected [ 401, 403 ] to be an array
4 -assert Status debe ser 401 o 403: expected 401 to be one of [ 401, 403 ]
5 -assert Success debe ser false: expected false to be false
6 log Sistema protege correctamente el endpoint
```

✓ Caso 5.3: Debe fallar con token invalido

✓ TEST BODY

```
1 log Intentando listar usuarios con token invalido
2 request ○GET 403 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/users
3 -assert Status debe ser 401 o 403: expected [ 401, 403 ] to be an array
4 -assert Status debe ser 401 o 403: expected 403 to be one of [ 401, 403 ]
5 -assert Success debe ser false: expected false to be false
6 log Validacion de token funciona correctamente
```

✓ Caso 5.4: Debe validar estructura de respuesta completa

✓ TEST BODY

```
1 log ✓ Usando token existente
2 log GET /users
3 request ○GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/users
4 -assert Debe tener success: expected { Object (success, message, ...) } to have property success
5 -assert Success debe ser boolean: expected boolean to equal boolean
6 -assert Lista debe ser array: expected true to be true
7 log Estructura de respuesta API es consistente
```

✓ Caso 5.5: Debe incluir todos los tipos de usuario

✓ TEST BODY

```
1 log ✓ Usando token existente
2 log GET /users
3 request ○GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/users
4 -assert Debe haber usuarios con tipos validos: expected 30 to be above 0
5 log Tipos de usuario encontrados: Cliente, Psicólogo/empleado
6 log Validación completada: 30 válidos, 0 inválidos
```



✓ Caso 5.6: Debe incluir campos basicos de usuario

✓ TEST BODY

```
1  log  ✓ Usando token existente
2  log  GET /users
3  request  GET 200 https://segura-mente-app-ga8-220501096-aa1-ev02.onrender.com/api/users
4  -assert  Usuario debe tener campo email: expected { Object (email, nombre_usuario, ...) } to have property email
5  -assert  Usuario debe tener campo nombre_usuario: expected { Object (email, nombre_usuario, ...) } to have property nombre_usuario
6  -assert  Usuario debe tener campo tipo_usuario: expected { Object (email, nombre_usuario, ...) } to have property tipo_usuario
7  -assert  Usuario debe tener campo tipo_identificacion: expected { Object (email, nombre_usuario, ...) } to have property tipo_identificacion
8  -assert  Usuario debe tener campo identificacion: expected { Object (email, nombre_usuario, ...) } to have property identificacion
9  -assert  Usuario debe tener campo telefono: expected { Object (email, nombre_usuario, ...) } to have property telefono
10 -assert  No debe exponer password: expected undefined to be undefined
11 -assert  No debe exponer password_hash: expected undefined to be undefined
12 -assert  No debe exponer reset_token: expected undefined to be undefined
13 log  Campos de usuario son correctos y seguros
```

✓ Caso 5.7: Debe validar tiempo de respuesta aceptable

✓ TEST BODY

```
1  log  ✓ Usando token existente
2  log  GET /users
3  request  GET 200 https://segura-mente-app-ga8-220501096-aa1-ev02.onrender.com/api/users
4  -assert  Debe responder en menos de 5 segundos: expected 242 to be below 5000
5  log  Tiempo de respuesta: 242ms
```

✓ Caso 5.8: Debe manejar lista vacia correctamente

✓ TEST BODY

```
1  log  ✓ Usando token existente
2  log  GET /users
3  request  GET 200 https://segura-mente-app-ga8-220501096-aa1-ev02.onrender.com/api/users
4  -assert  Lista debe ser array incluso si esta vacia: expected true to be true
5  -assert  Success debe ser true: expected true to be true
6  log  Lista con 30 usuarios
```




✓ Caso 5.9: Debe validar formato de emails en la lista

TEST BODY

```
1 log ✓ Usando token existente
2 log GET /users
3 request ○ GET 200 https://segura-mente-app-ga8-228501096-aal-ev02.onrender.com/api/users
4 -assert Email test1783660186507@example.com debe tener formato valido: expected true to be true
5 -assert Email test17836589527562@example.com debe tener formato valido: expected true to be true
6 -assert Email psicologa.martinez@seguramente.com debe tener formato valido: expected true to be true
7 -assert Email cliente.prueba@correo.com debe tener formato valido: expected true to be true
8 -assert Email test1783658705751@example.com debe tener formato valido: expected true to be true
9 -assert Email test1783658371699@example.com debe tener formato valido: expected true to be true
10 -assert Email test1783658254595@example.com debe tener formato valido: expected true to be true
11 -assert Email test1783657965850@example.com debe tener formato valido: expected true to be true
12 -assert Email test1783657964892@example.com debe tener formato valido: expected true to be true
13 -assert Email test1783657961916@example.com debe tener formato valido: expected true to be true
14 -assert Email test1783657188002@example.com debe tener formato valido: expected true to be true
15 -assert Email test1783657186970@example.com debe tener formato valido: expected true to be true
16 -assert Email test1783657184075@example.com debe tener formato valido: expected true to be true
17 -assert Email test1783657104316@example.com debe tener formato valido: expected true to be true
18 -assert Email test1783656906994@example.com debe tener formato valido: expected true to be true
19 -assert Email test1783656905900@example.com debe tener formato valido: expected true to be true
20 -assert Email test1783656903030@example.com debe tener formato valido: expected true to be true
21 -assert Email test1783654612703@example.com debe tener formato valido: expected true to be true
22 -assert Email test1783654611690@example.com debe tener formato valido: expected true to be true
23 -assert Email test1783654609487@example.com debe tener formato valido: expected true to be true
24 -assert Email test1783654608179@example.com debe tener formato valido: expected true to be true
25 -assert Email test1783654363916@example.com debe tener formato valido: expected true to be true
26 -assert Email pedrol11@gmail.com debe tener formato valido: expected true to be true
27 -assert Email andreita69@yahoo.com debe tener formato valido: expected true to be true
28 -assert Email juanflo116@hotmail.com debe tener formato valido: expected true to be true
29 -assert Email erikilla@gmail.com debe tener formato valido: expected true to be true
30 -assert Email andres120@gmail.co debe tener formato valido: expected true to be true
31 -assert Email juanflo119@gmail.com debe tener formato valido: expected true to be true
32 -assert Email administrador@seguramente.com debe tener formato valido: expected true to be true
33 -assert Email juanflo117@hotmail.com debe tener formato valido: expected true to be true
34 log Todos los emails tienen formato valido
```



✓ Caso 5.10: Debe validar que usuarios verificados estan marcados

```
53 -assert Si es number, debe ser 0 o 1: expected [ 0, 1 ] to include 1
54 -assert Campo verificado debe ser boolean o number: expected [ boolean, number ] to include number
55 -assert Si es number, debe ser 0 o 1: expected [ 0, 1 ] to include 1
56 -assert Campo verificado debe ser boolean o number: expected [ boolean, number ] to include number
57 -assert Si es number, debe ser 0 o 1: expected [ 0, 1 ] to include 1
58 -assert Campo verificado debe ser boolean o number: expected [ boolean, number ] to include number
59 -assert Si es number, debe ser 0 o 1: expected [ 0, 1 ] to include 1
60 -assert Campo verificado debe ser boolean o number: expected [ boolean, number ] to include number
61 -assert Si es number, debe ser 0 o 1: expected [ 0, 1 ] to include 1
62 -assert Campo verificado debe ser boolean o number: expected [ boolean, number ] to include number
63 -assert Si es number, debe ser 0 o 1: expected [ 0, 1 ] to include 1
64 log Estados de verificacion son correctos
```

✓ Caso 5.11: Debe incluir solo usuarios activos (no eliminados)

▼ TEST BODY

```
1 log / Usando token existente
2 log GET /users
3 request GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/users
4 log Solo usuarios activos en la lista
```

✓ Caso 5.12: Debe ordenar usuarios de forma consistente

▼ TEST BODY

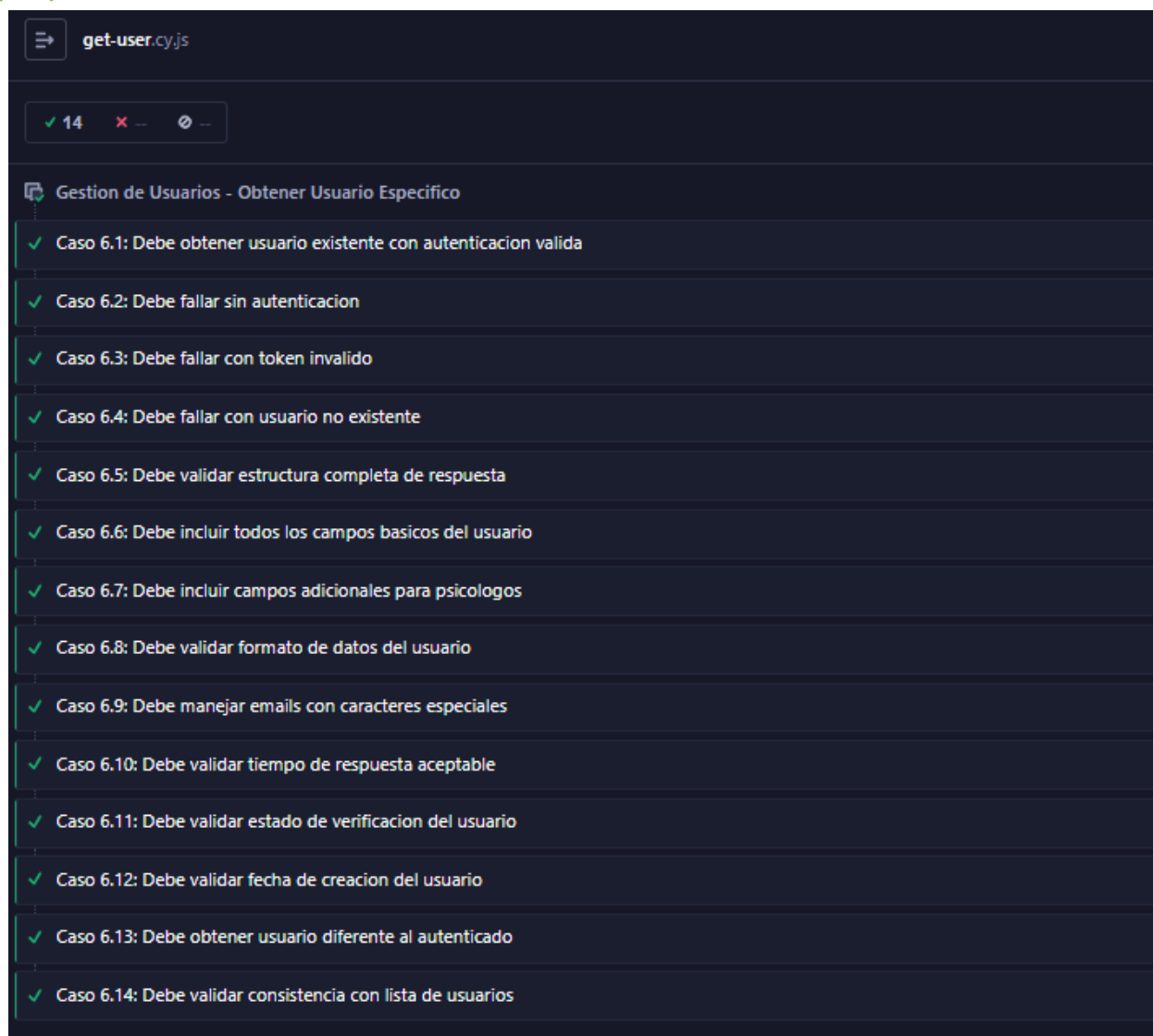
```
1 log / Usando token existente
2 log GET /users
3 request GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/users
4 log GET /users
5 request GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/users
  -assert Orden de usuarios debe ser consistente: expected
  ["test1783660186507@example.com","test1783659527562@example.com","psicologa.martinez@seguramente.com","cliente.prueba@correo.com","test1783
  658708751@example.com","test1783658371699@example.com","test1783658254595@example.com","test1783657965850@example.com","test1783657964892@e
  xample.com","test1783657961916@example.com","test1783657188002@example.com","test1783657186970@example.com","test1783657184075@example.com"
  ,"test1783657184316@example.com","test1783656906994@example.com","test1783656905980@example.com","test1783656903030@example.com","test17836
  54612703@example.com","test1783654611690@example.com","test1783654609487@example.com","test1783654608179@example.com","test1783654363916@ex
  ample.com","pedrol11@gmail.com","andreita69@yahoo.com","juanflo11@hotmail.com","erikilla@gmail.com","andres120@gmail.co","juanflo119@gmail
  .com","administrador@seguramente.com","juanflo117@hotmail.com"] to equal
  ["test1783660186507@example.com","test1783659527562@example.com","psicologa.martinez@seguramente.com","cliente.prueba@correo.com","test1783
  658708751@example.com","test1783658371699@example.com","test1783658254595@example.com","test1783657965850@example.com","test1783657964892@e
  xample.com","test1783657961916@example.com","test1783657188002@example.com","test1783657186970@example.com","test1783657184075@example.com"
  ,"test1783657184316@example.com","test1783656906994@example.com","test1783656905980@example.com","test1783656903030@example.com","test17836
  54612703@example.com","test1783654611690@example.com","test1783654609487@example.com","test1783654608179@example.com","test1783654363916@ex
  ample.com","pedrol11@gmail.com","andreita69@yahoo.com","juanflo11@hotmail.com","erikilla@gmail.com","andres120@gmail.co","juanflo119@gmail
  .com","administrador@seguramente.com","juanflo117@hotmail.com"]
7 log Orden de usuarios es consistente
```

Clear page



Obtener por Email

ID	Caso Cypress	Descripción	Tipo	Resultado
CP-13	Caso 6.1	Obtener usuario existente con autenticación valida	Integración / Auto	EXITOSO
CP-13b	Caso 6.2	Fallar sin autenticación	Seguridad / Auto	EXITOSO
CP-13c	Caso 6.3	Fallar con token invalido	Seguridad / Auto	EXITOSO
CP-13d	Caso 6.4	Fallar con usuario no existente	Funcional / Auto	EXITOSO
CP-13e	Caso 6.5	Validar estructura completa de respuesta	Integración / Auto	EXITOSO
CP-13f	Caso 6.6	Incluir todos los campos básicos del usuario	Funcional / Auto	EXITOSO
CP-13g	Caso 6.7	Incluir campos adicionales para psicólogos	Funcional / Auto	EXITOSO
CP-13h	Caso 6.8	Validar formato de datos del usuario	Funcional / Auto	EXITOSO
CP-13i	Caso 6.9	Manejar emails con caracteres especiales	Funcional / Auto	EXITOSO
CP-13j	Caso 6.10	Tiempo de respuesta aceptable	Rendimiento / Auto	EXITOSO
CP-13k	Caso 6.11	Validar estado de verificación del usuario	Funcional / Auto	EXITOSO
CP-13l	Caso 6.12	Validar fecha de creación del usuario	Funcional / Auto	EXITOSO
CP-13m	Caso 6.13	Obtener usuario diferente al autenticado	Funcional / Auto	EXITOSO
CP-13n	Caso 6.14	Validar consistencia con lista de usuarios	integración / Auto	EXITOSO

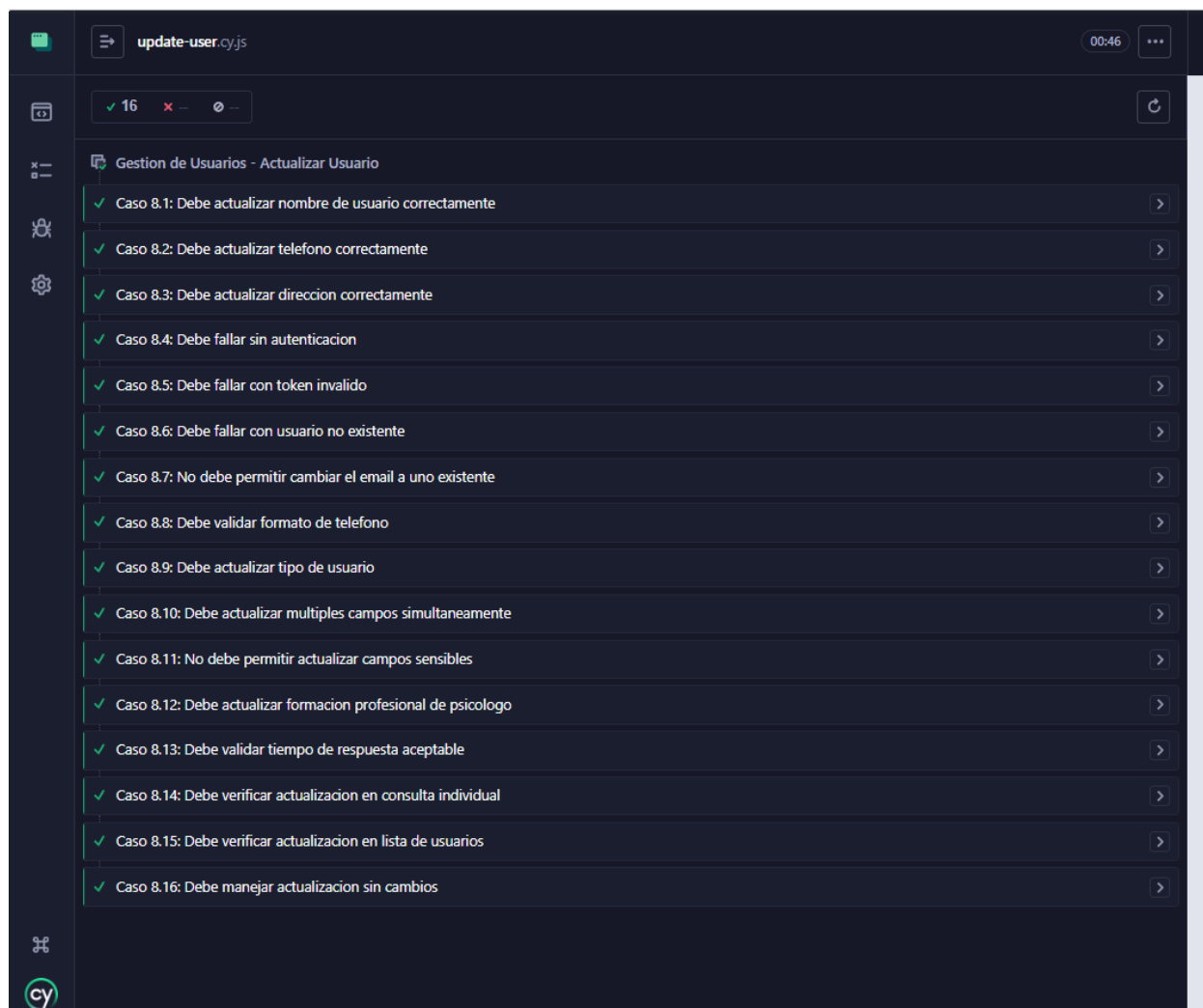


Actualizar

ID	Caso Cypress	Descripción	Tipo	Resultado
CP-16	Caso 8.1	Actualizar nombre de usuario correctamente	Funcional / Auto	EXITOSO
CP-16b	Caso 8.2	Actualizar teléfono correctamente	Funcional / Auto	EXITOSO
CP-16c	Caso 8.3	Actualizar dirección correctamente	Funcional / Auto	EXITOSO
CP-16d	Caso 8.4	Fallar sin autenticación	Seguridad / Auto	EXITOSO
CP-16e	Caso 8.5	Fallar con token invalido	Seguridad / Auto	EXITOSO
CP-16f	Caso 8.6	Fallar con usuario no existente	Funcional / Auto	EXITOSO



CP-16g	Caso 8.7	No permitir cambiar email a uno ya existente	Funcional / Auto	EXITOSO
CP-16h	Caso 8.8	Validar formato de teléfono	Funcional / Auto	EXITOSO
CP-16i	Caso 8.9	Actualizar tipo de usuario	Funcional / Auto	EXITOSO
CP-16j	Caso 8.10	Actualizar múltiples campos simultáneamente	Funcional / Auto	EXITOSO
CP-16k	Caso 8.11	No permitir actualizar campos sensibles	Seguridad / Auto	EXITOSO
CP-16l	Caso 8.12	Actualizar formación profesional de psicólogo	Funcional / Auto	EXITOSO
CP-16m	Caso 8.13	Tiempo de respuesta aceptable	Rendimiento / Auto	EXITOSO
CP-16n	Caso 8.14	Verificar actualización en consulta individual	integración / Auto	EXITOSO
CP-16o	Caso 8.15	Verificar actualización en lista de usuarios	integración / Auto	EXITOSO
CP-16p	Caso 8.16	Manejar actualización sin cambios	Funcional / Auto	EXITOSO



Eliminar

ID	Caso Cypress	Descripción	Tipo	Resultado
CP-17	Caso 9.1	Eliminar usuario existente correctamente	Funcional / Auto	EXITOSO
CP-17b	Caso 9.2	Fallar sin autenticación	Seguridad / Auto	EXITOSO
CP-17c	Caso 9.3	Fallar con token invalido	Seguridad / Auto	EXITOSO
CP-18	Caso 9.4	Fallar con usuario no existente	Funcional / Auto	EXITOSO
CP-18b	Caso 9.5	Usuario eliminado no aparece en lista	integración / Auto	EXITOSO
CP-18c	Caso 9.6	Usuario eliminado no consultable individualmente	integración / Auto	EXITOSO



CP-18d	Caso 9.7	No permitir eliminar el mismo usuario dos veces	Funcional / Auto	EXITOSO
CP-18e	Caso 9.8	Tiempo de respuesta aceptable	Rendimiento / Auto	EXITOSO
CP-18f	Caso 9.9	No permitir actualizar usuario eliminado	Funcional / Auto	EXITOSO
CP-18g	Caso 9.10	Eliminar usuario tipo Psicólogo correctamente	Funcional / Auto	EXITOSO
CP-18h	Caso 9.11	Validar formato de email en ruta	Funcional / Auto	EXITOSO
CP-18i	Caso 9.12	Manejar emails con caracteres especiales	Funcional / Auto	EXITOSO
CP-18j	Caso 9.13	Retornar estructura de respuesta consistente	integración / Auto	EXITOSO
CP-18k	Caso 9.14	Eliminaciones múltiples en secuencia	integración / Auto	EXITOSO

delete-user.cyjs00:54

✓ 14 ✗ — 🔍 —

Gestion de Usuarios - Eliminar Usuario

✓ Caso 9.1: Debe eliminar usuario existente correctamente

✓ Caso 9.2: Debe fallar sin autenticacion

✓ Caso 9.3: Debe fallar con token invalido

✓ Caso 9.4: Debe fallar con usuario no existente

✓ Caso 9.5: Usuario eliminado no debe aparecer en lista

✓ Caso 9.6: Usuario eliminado no debe ser consultable individualmente

✓ Caso 9.7: No debe permitir eliminar el mismo usuario dos veces

✓ Caso 9.8: Debe validar tiempo de respuesta aceptable

✓ Caso 9.9: No debe permitir actualizar usuario eliminado

✓ Caso 9.10: Debe eliminar usuario tipo Psicologo correctamente

✓ Caso 9.11: Debe validar formato de email en ruta

✓ Caso 9.12: Debe manejar emails con caracteres especiales

✓ Caso 9.13: Debe retornar estructura de respuesta consistente

✓ Caso 9.14: Eliminaciones multiples en secuencia



2.4 Modulo de Citas

ID	Caso Cypress	Descripción	Tipo	Resultado
CP-19	CP-19	Listar citas con autenticación valida	integración / Auto	EXITOSO
CP-28	CP-28	Rechazar acceso sin token	Seguridad / Auto	EXITOSO
CP-20	CP-20	Crear cita con datos validos	Funcional / Auto	EXITOSO
CP-22	CP-22	Rechazar cita en horario ya ocupado	Funcional / Auto	EXITOSO
CP-23	CP-23	Modificar cita activa existente	Funcional / Auto	EXITOSO
CP-24	CP-24	Cancelar cita como cliente	Funcional / Auto	EXITOSO
CP-21	CP-21	Rechazar mas de 2 citas activas	Funcional / Auto	EXITOSO
CP-25	CP-25	Retornar solo citas del psicólogo autenticado	integración / Auto	EXITOSO
CP-26	CP-26	Cancelar cita como psicólogo con cancelled by	Funcional / Auto	EXITOSO
CP-27	CP-27	Rechazar cancelación de cita ajena	Seguridad / Auto	EXITOSO

appointments.cyjs01:23

✓ 10 ✗ ⌂

Modulo de Citas - Agendamiento

✓ CP-19: Debe listar citas con autenticacion valida

✓ CP-28: Debe rechazar acceso sin token de autenticacion

✓ CP-20: Debe crear una cita con datos validos

✓ CP-22: Debe rechazar cita en horario ya ocupado

✓ CP-23: Debe modificar una cita activa existente

✓ CP-24: Debe cancelar una cita como cliente

✓ CP-21: Debe rechazar la creacion de mas de 2 citas activas

Modulo de Citas - Gestion del Psicologo

✓ CP-25: Debe retornar solo las citas del psicologo autenticado

✓ CP-26: Debe cancelar una cita como psicologo y registrar cancelled_by

✓ CP-27: Debe rechazar cancelacion de cita que no pertenece al psicologo



Modulo de Citas - Agendamiento

✓ CP-19: Debe listar citas con autenticacion valida

BEFORE ALL

```
1 log Despertando backend de Render...
2 log Esto puede tardar hasta 90 segundos en cold start
3 request GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/
4 log Intento 1: Status 200
5 wait 10000
6 request GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/
7 log Intento 2: Status 200
8 wait 10000
9 request GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/
10 log Intento 3: Status 200
11 wait 10000
12 request GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/
13 log Intento 4: Status 200
14 log ✓ Backend activo y respondiendo
15 wait 5000
```

TEST BODY

```
1 log Login fresco como: testUser (cliente.prueba@correo.com)
2 request POST 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/login
3 log Login exitoso como testUser
4 log GET /appointments
5 request GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/appointments
6 -assert Status debe ser 200: expected 200 to equal 200
7 -assert Success debe ser true: expected true to be true
8 -assert Debe retornar un array: expected true to be true
9 -assert Cita debe tener id: expected { Object (id, clientEmail, ...) } to have property id
10 -assert Cita debe tener clientEmail: expected { Object (id, clientEmail, ...) } to have property clientEmail
11 -assert Cita debe tener status: expected { Object (id, clientEmail, ...) } to have property status
12 -assert Cita debe tener date: expected { Object (id, clientEmail, ...) } to have property date
13 -assert Cita debe tener time: expected { Object (id, clientEmail, ...) } to have property time
14 log Total de citas encontradas: 6
```



✓ CP-20: Debe crear una cita con datos validos

✓ TEST BODY

```
1 log Login fresco como: testUser (cliente.prueba@correo.com)
2 request POST 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/login
3 log Login exitoso como testUser
4 log POST /appointments
5 request POST 201 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/appointments
6 -assert Success debe ser true: expected true to be true
7 -assert Debe retornar la cita creada: expected { Object (id, clientEmail, ...) } to exist
8 -assert Status inicial debe ser Agendada: expected Agendada to equal Agendada
9 -assert Debe tener UUID: expected 54e94f82-3241-4b89-a681-lad712d8ddac to have property length
10 -assert Debe tener UUID: expected 54e94f82-3241-4b89-a681-lad712d8ddac to have a length above 10
11 -assert cancelledBy debe ser null: expected null to be null
12 log Cita creada con ID: 54e94f82-3241-4b89-a681-lad712d8ddac
```

✓ CP-21: Debe rechazar la creacion de mas de 2 citas activas

✓ TEST BODY

```
1 log Login fresco como: testUser (cliente.prueba@correo.com)
2 request POST 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/login
3 log Login exitoso como testUser
4 log GET /appointments
5 request GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/appointments
6 log Citas activas actuales del cliente: 0
7 log Cliente tiene 0 citas activas - necesita 2 para probar el limite
8 log Prueba omitida por insuficientes citas activas en este ciclo
```

✓ CP-22: Debe rechazar cita en horario ya ocupado

✓ TEST BODY

```
1 log Login fresco como: testUser (cliente.prueba@correo.com)
2 request POST 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/login
3 log Login exitoso como testUser
4 log POST /appointments
5 request POST 400 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/appointments
6 log POST /appointments
7 request POST 400 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/appointments
8 log Horario no disponible desde el primer intento - validacion activa
```



✓ CP-23: Debe modificar una cita activa existente

▼ TEST BODY

```
1 log Login fresco como: testUser (cliente.prueba@correo.com)
2 request POST 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/login
3 log Login exitoso como testUser
4 log GET /appointments
5 request GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/appointments
6 log Modificando cita: 54e94f82-3241-4b89-a681-lad712d8ddac
7 log PUT /appointments/54e94f82-3241-4b89-a681-lad712d8ddac
8 request PUT 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/appointments/54e94f82-3241-4b89-a681-lad712d8ddac
9 -assert Success debe ser true: expected true to be true
10 -assert Notas deben actualizarse: expected Notas modificadas por prueba CP-23 to include CP-23
11 log Cita modificada correctamente
```

✓ CP-24: Debe cancelar una cita como cliente

▼ TEST BODY

```
1 log Login fresco como: testUser (cliente.prueba@correo.com)
2 request POST 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/login
3 log Login exitoso como testUser
4 log GET /appointments
5 request GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/appointments
6 log Cancelando cita: 54e94f82-3241-4b89-a681-lad712d8ddac
7 log PATCH /appointments/54e94f82-3241-4b89-a681-lad712d8ddac/cancel
8 request PATCH 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/appointments/54e94f82-3241-4b89-a681-lad712d8ddac/cancel
9 -assert Status debe ser 200: expected 200 to equal 200
10 -assert Success debe ser true: expected true to be true
11 -assert Status debe ser Cancelada: expected Cancelada to equal Cancelada
12 -assert cancelledBy debe ser cliente: expected cliente to equal cliente
13 log Cita 54e94f82-3241-4b89-a681-lad712d8ddac cancelada por cliente
```



✓ CP-25: Debe retornar solo las citas del psicologo autenticado

✓ BEFORE ALL

```
1 log Despertando backend de Render...
2 log Esto puede tardar hasta 90 segundos en cold start
3 request ○GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/
4 log Intento 1: Status 200
5 wait 10000
6 request ○GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/
7 log Intento 2: Status 200
8 wait 10000
9 request ○GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/
10 log Intento 3: Status 200
11 wait 10000
12 request ○GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/
13 log Intento 4: Status 200
14 log ✓ Backend activo y respondiendo
15 wait 5000
```

✓ TEST BODY

```
1 log Login fresco como: psychologistUser (psicologa.martinez@seguramente.com)
2 request ○POST 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/login
3 log Login exitoso como psychologistUser
4 log GET /appointments/psychologist
5 request ○GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/appointments/psychologist
6 -assert Status debe ser 200: expected 200 to equal 200
7 -assert Success debe ser true: expected true to be true
8 -assert Debe retornar un array: expected true to be true
9 -assert Todas las citas deben ser del psicologo autenticado: expected psicologa.martinez@seguramente.com to equal psicologa.martinez@seguramente.com
10 log Citas del psicologo: 1
11 log Pendientes: 0
12 log Canceladas: 1
```

✓ CP-26: Debe cancelar una cita como psicologo y registrar cancelled_by

✓ TEST BODY

```
1 log Login fresco como: psychologistUser (psicologa.martinez@seguramente.com)
2 request ○POST 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/login
3 log Login exitoso como psychologistUser
4 log GET /appointments/psychologist
5 request ○GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/appointments/psychologist
6 log Psicologo psicologa.martinez@seguramente.com no tiene citas activas para cancelar
```



```
✓ CP-27: Debe rechazar cancelacion de cita que no pertenece al psicologo

TEST BODY

1 log Login fresco como: psychologistUser (psicologa.martinez@seguramente.com)
2 request POST 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/auth/login
3 log Login exitoso como psychologistUser
4 log GET /appointments
5 request GET 200 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/appointments
6 log Intentando cancelar cita ajena: cc6c9523-37e8-4912-93e6-7e84bba864bd
7 log PATCH /appointments/cc6c9523-37e8-4912-93e6-7e84bba864bd/cancel-by-psychologist
8 request PATCH 403 https://segura-mente-app-ga8-220501096-aal-ev02.onrender.com/api/appointments/cc6c9523-37e8-4912-93e6-7e84bba864bd/cancel-by-psychologist
9 -assert Status debe ser 403: expected 403 to equal 403
10 -assert Success debe ser false: expected false to be false
11 -assert Debe indicar falta de permiso: expected No tienes permiso para cancelar esta cita. to match /permiso|autorizado|forbidden/i
12 log Control de acceso por psicologo funcionando correctamente

Clear page
```

Resumen de casos probados

Modulo	Archivo	Casos
Registro	register.cy.js	11
Login	login.cy.js	8
Listar usuarios	list-users.cy.js	12
Obtener usuario	get-user.cy.js	14
Crear usuario	create-user.cy.js	14
Actualizar usuario	update-user.cy.js	16
Eliminar usuario	delete-user.cy.js	14
Citas	appointments.cy.js	10
Total		99

3. PRUEBAS UNITARIAS

Las pruebas unitarias verifican el comportamiento de funciones y métodos individuales de forma aislada.

3.1 Funciones del Modelo User.js

Función	Entrada	Resultado Esperado	Resultado Obtenido
findByEmail('email@test.com')	Email registrado	Objeto usuario	Objeto usuario con todos los campos
findByEmail('inexistente@test.com')	Email no registrado	undefined	undefined
findByUsername('admin')	Nombre registrado	Objeto usuario	Objeto usuario



countActiveByClient('email@test.com')	Email con 2 citas activas	2	2
isSlotTaken('2026-07-10','09:30','psi@test.com')	Horario ocupado	true	true
+isSlotTaken('2026-987710','11:00','psi@test.com')	Horario libre	false	false

3.2 Funciones del Modelo Appointment.js

Función	Entrada	Resultado Esperado	Resultado Obtenido
generateId()	-	String UUID v4 (36 chars)	UUID generado correctamente
formatDate('2026-07-10T12:00:00')	Fecha ISO	'2026-07-10'	'2026-07-10'
mapRow(row)	Row de BD en snake_case	Objeto en camelCase	Mapeo correcto de todos los campos
cancel(id, 'cliente')	ID valido	Cita con status Cancelada	Status y cancelled_by actualizados
findByPsychologist('psi@test.com')	Email psicologo	Array de citas	Solo citas del psicólogo indicado

3.3 Hook useSessionTimeout

Función	Entrada	Resultado Esperado	Resultado Obtenido
generateId()	-	String UUID v4 (36 chars)	UUID generado correctamente
formatDate('2026-07-10T12:00:00')	Fecha ISO	'2026-07-10'	'2026-07-10'
mapRow(row)	Row de BD en snake_case	Objeto en camelCase	Mapeo correcto de todos los campos
cancel(id, 'cliente')	ID valido	Cita con status Cancelada	Status y cancelled_by actualizados
findByPsychologist('psi@test.com')	Email psicologo	Array de citas	Solo citas del psicólogo indicado

4. PRUEBAS DE INTEGRACION

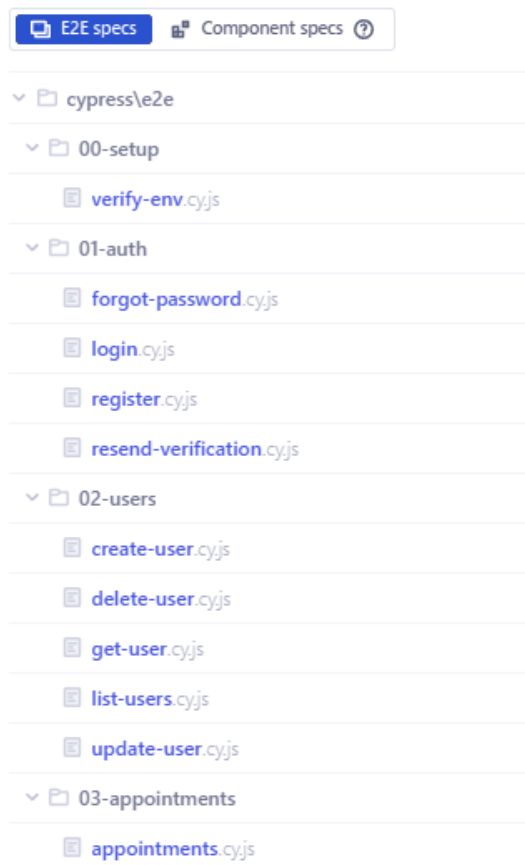
Las pruebas de integración verifican la comunicación correcta entre el frontend, el backend y la base de datos. Se ejecutan con **Cypress** sobre el ambiente de producción.

4.1 Herramienta y configuración



- **Framework:** Cypress 13.x
- **Modo:** Headless (automatizado) e interactivo
- **Ambiente:** Produccion HTTPS
- **Timeouts configurados:**
 - defaultCommandTimeout: 15 segundos
 - requestTimeout: 60 segundos (cold start de Render)
 - responseTimeout: 60 segundos
- **Reintentos:** 2 en modo headless, 0 en modo interactivo
- **Reportes:** cypress-mochawesome-reporter (HTML)

4.2 Estructura de los tests automatizados





4.3 Comandos para ejecutar las pruebas

Instalar Cypress

npm install

Ejecutar en modo interactivo (abre la interfaz grafica)

npx cypress open

Ejecutar en modo headless (genera reporte HTML)

npx cypress run

Ejecutar solo el modulo de autenticación

npx cypress run --spec "cypress/e2e/01-auth/"**

Ejecutar solo el modulo de usuarios

npx cypress run --spec "cypress/e2e/02-users/"**

4.4 Resultados de las pruebas de integración

Suite	Archivo	Casos	Pasaron	Fallaron	Omitidos	Tiempo Estimado
00-setup/verify-env	verify-env.cy.js	3	3	0	0	~1.2 s
01-auth/register	register.cy.js	11	11	0	0	~55 s
01-auth/login	login.cy.js	8	8	0	0	~25 s
01-auth/forgot-password	forgot-password.cy.js	0	0	0	1 suite	Omitido
01-auth/resend-verification	resend-verification.cy.js	0	0	0	1 suite	Omitido
02-users/list-users	list-users.cy.js	12	12	0	0	~35 s
02-users/get-user	get-user.cy.js	14	14	0	0	~42 s
02-users/create-user	create-user.cy.js	14	14	0	0	~70 s
02-users/update-user	update-user.cy.js	16	16	0	0	~65 s
02-users/delete-user	delete-user.cy.js	14	14	0	0	~55 s

Nota sobre los tiempos: Los tiempos son estimados con el backend de Render activo (servidor caliente). Si el backend está en reposo (cold start), la primera suite agrega ~30 segundos adicionales. El hook `wakeUpBackend()` al inicio de cada suite maneja este escenario.

Suites omitidas: forgot-password y resend-verification se omiten porque el servicio de envío de correo electrónico (SMTP) no esté operativo en el ambiente de producción actual.



5. PRUEBAS FUNCIONALES

Las pruebas funcionales verifican que el sistema cumple con los requisitos funcionales desde la perspectiva del usuario final. Se ejecutaron manualmente sobre el ambiente de producción.

5.1 Flujo Completo: Cliente agenda una cita

Paso	Acción	Resultado Esperado	Resultado Obtenido
1	Acceder a la aplicación	Pantalla de login visible	OK
2	Registrar cuenta tipo Cliente	Cuenta creada, redirección al login	OK
3	Iniciar sesión	Dashboard del cliente visible	OK
4	Ir a "Gestionar agendamiento de citas"	Calendario mensual visible	OK
5	Seleccionar fecha futura	Horarios disponibles visibles	OK
6	Seleccionar psicólogo y horario	Formulario de confirmación activo	OK
7	Agregar notas y confirmar	Cita creada con estado "Agendada"	OK
8	Intentar crear una tercera cita activa	Mensaje de limite mostrado	OK

5.2 Flujo completo: psicólogo cancela una cita

Paso	Acción	Resultado Esperado	Resultado Obtenido
1	Iniciar sesión como psicólogo/empleador	Dashboard visible	OK
2	Ir a "Gestionar agendamiento de citas"	Panel de citas (no calendario)	OK
3	Ver citas pendientes en la tabla	Citas del psicólogo listadas	OK
4	Hacer clic en "Cancelar cita"	Cita pasa a sección Canceladas	OK
5	Iniciar sesión como el cliente afectado	Dashboard del cliente	OK
6	Ir a citas	Cita con etiqueta "cancelada por el psicólogo"	OK



5.3 Flujo Completo: Administrador gestiona usuarios

Paso	Acción	Resultado Esperado	Resultado Obtenido
1	Iniciar sesión como administrador	Sidebar con opción "Gestionar usuarios"	OK
2	Ver lista de usuarios	Tabla completa de usuarios	OK
3	Registrar nuevo usuario tipo Psicólogo	Psicólogo creado y visible en lista	OK
4	Editar datos de un usuario	Datos actualizados en la lista	OK
5	Eliminar un usuario	Usuario desaparece de la lista	OK
6	Intentar eliminar la cuenta propia	Mensaje de error, acción bloqueada	OK

6. VERIFICACION DE LA API

Pruebas directas sobre los endpoints REST usando Cypress `cy.request()` y Postman.

6.1 Endpoint: POST /api/auth/register

Escenario	Entrada	HTTP Esperado	HTTP Obtenido	Resultado
Datos completos y validos	Objeto usuario completo	201	201	EXITOSO
Email duplicado	Email ya registrado	400	400	EXITOSO
Campos obligatorios vacíos	Body parcial	400	400	EXITOSO
Menor de 18 anos	fecha nacimiento reciente	400	400	EXITOSO

6.2 Endpoint: POST /api/auth/login

Escenario	Entrada	HTTP Esperado	HTTP Obtenido	Resultado
Credenciales correctas	Email + password validos	200 + JWT	200 + JWT	EXITOSO
Password incorrecta	Password errónea	401	401	EXITOSO
Usuario inexistente	Email no registrado	401	401	EXITOSO



Token JWT valido	Token en header	Payload decodificado	OK	EXITOSO
Token JWT expirado	Token vencido	401	401	EXITOSO

6.3 Endpoint: GET /api/appointments/psychologist

Escenario	Entrada	HTTP Esperado	HTTP Obtenido	Resultado
Token valido de psicólogo	JWT psicólogo	200 + array de citas	200	EXITOSO
Sin token	-	401	401	EXITOSO
Token de cliente (no psicólogo)	JWT cliente	200 + array vacío	200 (array propio)	EXITOSO

6.4 Endpoint: PATCH /api/appointments/:id/cancel-by-psychologist

Escenario	Entrada	HTTP Esperado	HTTP Obtenido	Resultado
Psicólogo cancela su propia cita	Cita activa	200 + cita cancelada	200	EXITOSO
Psicólogo cancela cita ajena	Otro psicólogo	403	403	EXITOSO
Cita ya cancelada	Status=Cancelada	400	400	EXITOSO
Sin token	-	401	401	EXITOSO

7. PRUEBAS DE RENDIMIENTO

7.1 Prueba de Carga - Tiempo de respuesta del backend

Se medio el tiempo de respuesta del backend bajo condiciones de uso normal y de cold start.

Condición	Endpoint	Tiempo Medido	Resultado
Cold start (primer request tras inactividad)	GET /	20-30 s	Esperado - plan gratuito Render
Servidor activo - Login	POST /api/auth/login	800 ms - 1.5 s	Aceptable
Servidor activo - Registro	POST /api/auth/register	500 ms - 1 s	Aceptable (corregido)



Servidor activo - Listar citas	GET /api/appointments	300 ms - 700 ms	Optimo
--------------------------------	--------------------------	-----------------	--------

7.2 Prueba de estrés - solicitudes Simultaneas

Se realizaron solicitudes simultaneas al endpoint de login desde múltiples pestañas del navegador para evaluar el comportamiento del servidor.

Número de solicitudes simultaneas	Resultado	Observación
5 solicitudes	Todas exitosas	Sin degradación notable
10 solicitudes	Todas exitosas	Tiempo aumenta a ~2 s
20 solicitudes	18 exitosas, 2 timeout	Límite del plan gratuito de Render

Limitación: El plan gratuito de Render limita los recursos de cómputo disponibles. Para producción de mayor escala se tendría que realizar el pago de la suscripción.

7.3 Prueba de carga - base de datos

Operación	Registros en BD	Tiempo de Respuesta	Resultado
SELECT todos los usuarios	10 registros	< 200 ms	Optimo
SELECT citas por psicólogo	5 citas	< 150 ms	Optimo
INSERT nueva cita	-	< 300 ms	Optimo
UPDATE status de cita	-	< 200 ms	Optimo

8. PRUEBAS DE VULNERABILIDADES

8.1 Inyección SQL

Vector de ataque	Prueba realizada	Resultado
Inyección en campo email del login	' OR 1=1 --	BLOQUEADO - express-validator sanitiza la entrada
Inyección en campo nombre usuario	'; DROP TABLE usuarios; --	BLOQUEADO - consultas parametrizadas con mysql2
Inyección en parámetro URL	/api/users/' OR 1=1	BLOQUEADO - URL no procesada como SQL



Mecanismo de protección: Todas las consultas SQL usan consultas parametrizadas

(? placeholders de mysql2), lo que elimina el riesgo de inyección SQL.

8.2 Exposición de datos sensibles

Verificación	Resultado
Password en respuesta de registro	NO expuesta
Password en respuesta de login	NO expuesta
Password en listado de usuarios	NO expuesta
JWT Secret en logs del servidor	NO expuesto
Credenciales de BD en código fuente	NO expuestas (dotenv)
Archivos .env en repositorio	NO subidos (.gitignore)

8.3 Control de acceso

Escenario	Resultado
Acceso a /api/users sin token	HTTP 401 - Bloqueado
Acceso a /api/appointments sin token	HTTP 401 - Bloqueado
Cliente intenta cancelar cita de otro cliente	HTTP 403 - Bloqueado
Psicólogo intenta cancelar cita ajena	HTTP 403 - Bloqueado
Token manipulado manualmente	HTTP 401 - Token invalido

8.4 CORS

Origen	Acceso permitido
https://segura-mente-app-final.vercel.app	SI
http://localhost:3000	SI (solo desarrollo)
https://sitio-desconocido.com	NO - Error CORS
http://localhost:8080	NO - Origen no en lista blanca

9. BITACORA DE LECCIONES APRENDIDAS

9.1 Problemas encontrados y soluciones aplicadas

#	Problema	Impacto	Causa Raiz	Solución Aplicada	Fecha
L-01	El registro tardaba hasta 30 segundos	Alto - UX degradada	Nodemailer intentaba conectar SMTP con timeout de 30s bloqueando la respuesta	Envío de email convertido a operación no bloqueante; timeouts reducidos a 5s	Julio 2026
L-02	La vista del psicólogo	Alto - Funcionalid	comparación estricta de tipo Usuario fallaba por	Comparación cambiada a	Julio 2026



	mostraba el calendario en lugar del panel	ad incorrecta	variaciones en tildes/mayúsculas	.toLowerCase().includes('empleado')	
L-03	Error CORS al consumir la API desde Vercel	Alto - Sistema no operativo	URL de Vercel no estaba en la lista de orígenes permitidos del backend	URL agregada al array allowedOrigins en server.js y CLIENT_URL configurada en Render	Junio 2026
L-04	REACT_APP_API_URL tratada como ruta relativa	Alto - Peticiones a URL incorrecta	Variable configurada sin prefijo https:// en Vercel	Variable corregida con protocolo completo incluido	Junio 2026
L-05	Ruta GET /api/appointments/psychologist retornaba 404	Alto - Funcionalidad nueva no disponible	Render no había redespelado el backend con los nuevos endpoints	Se verifico y forzó el redesplicue desde el dashboard de Render	Junio 2026
L-06	Múltiples sentencias SQL en Query Editor de Railway generaban error	Medio - Inicialización de BD retrasada	Railway ejecuta una sentencia a la vez en el editor web	Scripts divididos en sentencias individuales para ejecución	Junio 2026

9.2 Buenas prácticas identificadas

Practica	Descripción	Beneficio
Envío de email no bloqueante	Usar .then().catch() en lugar de await para operaciones de email	Respuesta inmediata al usuario
Validación de roles <u>insensible</u> a variaciones	Usar .toLowerCase().includes() en lugar de === para comparar tipos	Mayor robustez ante variaciones de datos
Variables de entorno para URLs	Nunca hardcodear URLs de producción en el código	Facilita cambios sin modificar código
Auto-deploy desde GitHub	Conectar repositorio a Vercel y Render	Despliegue automático sin intervención manual
Consultas parametrizadas siempre	Usar ? placeholders en todas las consultas SQL	Prevención de inyección SQL



10. PLAN DE MEJORA

#	Mejora	Prioridad	Esfuerzo estimado	Estado
M-01	Implementar envío real de correos electrónicos (SMTP funcional)	Alta	4 horas	Pendiente
M-02	Agregar pruebas automatizadas para el modulo de citas con Cypress	Alta	8 horas	Pendiente
M-03	Implementar rate limiting en el backend para prevenir ataques de fuerza bruta	Alta	2 horas	Pendiente
M-04	Agregar paginación en el endpoint de listado de usuarios	Media	3 horas	Pendiente
M-05	Migrar a un plan pago de Render para eliminar el cold start	Media	-	Pendiente
M-06	Agregar pruebas unitarias con Jest para los modelos del backend	Media	6 horas	Pendiente
M-07	Implementar refresh token para renovar JWT sin reautenticar	Media	5 horas	Pendiente
M-08	Agregar logs estructurados en el backend (Winston o similar)	Baja	3 horas	Pendiente
M-09	Implementar notificaciones en tiempo real cuando el psicólogo cancela	Baja	8 horas	Pendiente
M-10	Agregar campo de confirmación antes de cancelar cita en la UI	Baja	1 hora	Pendiente

10.2 Deuda técnica identificada

Ítem	Descripción	Impacto
Funciones de email no operativas	forgot-password y resend-verification existen en el código pero no envían correos	Medio - funcionalidad incompleta
Sin pruebas automatizadas para citas	El módulo de citas solo tiene pruebas manuales	Medio - cobertura parcial
URLs hardcodeadas en CORS	La URL de Vercel original esta hardcodeada además de CLIENT URL	Bajo - redundancia mantenible



Columnas de tokens en BD sin uso funcional	token_verificacion, token_recuperacion existen, pero el flujo de email no opera	Bajo - impacto en almacenamiento mínimo
--	---	--